

★ COMPLETE PREPARATION GUIDE

Everything You Need to Know Before You Begin

A comprehensive, topic-by-topic guide to the skills, concepts, and resources you need before starting the Principal Strategist 90-Day Arbitrage Training Program. Every section includes free online learning links.

- Python 3.8+
- Type Annotations
- Exception Handling
- SQLite
- HTTP / JSON
- Command Line
- Standard Library
- Financial Concepts

📄 TABLE OF CONTENTS

1. Python Fundamentals
2. Type Annotations
3. Exception Handling
4. SQL & SQLite
5. HTTP & JSON
6. Command Line Basics
7. Standard Library Tour
8. Financial Concepts
9. CMA / SOCPA / IFRS
10. Environment Setup
- ✅ Final Checklist

1 / 10

Python Fundamentals

You must be comfortable reading and writing Python at an intermediate level. The simulator will test every function you write through AST audit and hidden tests.



What You Need to Know

TOPIC	REQUIRED LEVEL	TESTED IN
Variables & data types	★★★	Day 1+
Functions (def, return)	★★★★	Day 1+
Lists, dicts, tuples, sets	★★★★	Day 1+
Loops (for, while)	★★★★	Day 1+
Conditionals (if/elif/else)	★★★★	Day 1+
List comprehensions	★★★	Day 6+
Slicing & string formatting	★★★	Day 1+
File I/O	★★★★	Day 4+
Modules & imports	★★★★	Day 1+



Free Learning Resources



Official Python Tutorial

python.org — The definitive starting point



LearnPython.org

Interactive browser-based Python tutorials



Real Python

In-depth tutorials for all skill levels



W3Schools Python

Quick reference with interactive examples



**Codecademy Python 3**

Structured course with exercises

**Automate the Boring Stuff**

Free book with practical Python projects



Concept Deep-Dive: Lists & Slicing

You will frequently work with sliding windows of price data. List slicing is essential.

```
# Sliding window of last 5 prices prices = [100.0, 101.5, 102.3, 101.8, 103.2, 104.0,
103.5] # Last 5 elements window = prices[-5:] # [101.8, 103.2, 104.0, 103.5] # SMA(5)
calculation sma_5 = sum(prices[-5:]) / 5 # List comprehension for SMA values def sma(data,
window): return [sum(data[i-window:i])/window if i >= window else None for i in
range(len(data)+1)][1:]
```

DAY 1

AST: NESTED_LOOP

⚠ Common Mistakes That Will Fail You

✗ **Using eval() or exec():** The AST auditor flags these as critical violations (-40 pts). Never use them.

✗ **O(n²) nested loops:** for i in range(len(p)): for j in range(len(p)): triggers a warning (-10 pts). Use sliding windows instead.

✗ **Division by zero:** Always guard a / b with if b != 0: or use try/except ZeroDivisionError.

2 / 10

Type Annotations (PEP 484)

The AST auditor checks that every function parameter and return value has a type hint. Missing annotations are recorded as INFO-level findings. While they don't deduct points, the Principal Strategist expects professional-grade code.



The Syntax

```
# Function with type annotations def calculate_sma( prices: List[float], # parameter type
window: int # parameter type ) -> List[Optional[float]]: # return type """Calculate Simple
Moving Average.""" pass
```

Always import from typing:

```
from typing import List, Dict, Optional, Tuple, Union, Any
```



Learning Resources



Official typing Module Docs

python.org — Complete reference



mypy Cheat Sheet

Quick reference for common type patterns



Real Python: Type Checking Guide

Tutorial with examples



PEP 484 — Type Hints

The original specification



Types You'll Use Most

```
from typing import List, Dict, Optional, Tuple, Union # Basic types name: str = "BTC/USD"
price: float = 67306.92 count: int = 30 active: bool = True # Collections prices:
List[float] = [100.0, 101.5, 102.3] orderbook: Dict[str, Any] = {"bids": [], "asks": []} #
```

```
Optional (can be None) sma_value: Optional[float] = None # Tuple (fixed-size, possibly  
mixed types) trade: Tuple[str, float, float] = ("BTC/USD", 1.0, 50000.0)
```

3 / 10

Exception Handling

This is the most critical skill. The AST auditor gives a -40 CRITICAL violation for every network call without a try-except wrapper. Two such violations = automatic 0 for the day.

⚠ The Rule

Every single `urllib.request` call must be wrapped in `try/except`. The exchange can go down, return rate limits, or send malformed data. Your pipeline must survive all of it.

```
import urllib.request import urllib.error def fetch_orderbook(url: str) -> Optional[dict]:
try: with urllib.request.urlopen(url, timeout=5) as resp: return
json.loads(resp.read().decode()) except urllib.error.HTTPError as e: if e.code == 429:
time.sleep(1) # rate-limited, back off return None except urllib.error.URLError as e:
print(f"Network error: {e.reason}") return None except json.JSONDecodeError as e:
print(f"Bad JSON: {e}") return None except socket.timeout: print("Connection timed out")
return None
```

CRITICAL

DAY 1+

🔗 Learning Resources



Official Python Errors & Exceptions

[python.org — Complete tutorial](https://docs.python.org/3/library/exceptions.html) →

Real Python: Exception Handling

[try/except/else/finally explained](https://realpython.com/python-exceptions/) →

Built-in Exception List

[python.org — All exception classes](https://docs.python.org/3/library/exceptions.html) →

urllib HOWTO

[Including error handling patterns](https://docs.python.org/3/library/urllib.html) →

Exceptions You Must Handle

EXCEPTION	WHEN IT OCCURS	EXAMPLE
<code>urllib.error.URLError</code>	DNS failure, connection refused, network down	Exchange server not running
<code>urllib.error.HTTPError</code>	Server returned 4xx or 5xx	Rate limit (429), bad request (400)
<code>json.JSONDecodeError</code>	Response body is not valid JSON	Server returns HTML error page
<code>socket.timeout</code>	Connection took too long	Server overloaded
<code>ZeroDivisionError</code>	Division by zero in SMA/ratios	Empty price window
<code>sqlite3.Error</code>	Database operation fails	Constraint violation, disk full

4 / 10

SQL & SQLite

Every trade, calculation, and audit event is logged to a double-entry SQLite database. You must be comfortable with basic CRUD operations, aggregation, and transaction integrity.

SQLite in Python

SQLite is embedded in Python's standard library — zero setup required. The database is a single file (training_ledger.db).

```
import sqlite3 # Connect (creates file if it doesn't exist) conn =
sqlite3.connect("training_ledger.db") cursor = conn.cursor() # Create table
cursor.execute('''CREATE TABLE IF NOT EXISTS trades ( id TEXT PRIMARY KEY, day INTEGER,
symbol TEXT, side TEXT, price REAL, quantity REAL, total REAL, fee REAL, timestamp TEXT
)''') # Insert cursor.execute("INSERT INTO trades VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)",
(order_id, day, symbol, side, price, qty, total, fee, ts)) # Query cursor.execute("SELECT
COUNT(*) FROM trades WHERE day=?", (day,)) count = cursor.fetchone()[0] # Always commit and
close conn.commit() conn.close()
```

The Ledger Schema

Your solutions will interact with these tables:

```
-- Account balances (9 seeded accounts) CREATE TABLE accounts ( code TEXT PRIMARY KEY, --
e.g. '1000', '1100' name TEXT NOT NULL, -- e.g. 'Cash - USD' type TEXT NOT NULL, --
'asset', 'revenue', 'expense' balance REAL DEFAULT 0.0 ); -- Trade Log (every executed
trade) CREATE TABLE trades ( id TEXT PRIMARY KEY, day INTEGER, symbol TEXT, side TEXT,
price REAL, quantity REAL, total REAL, fee REAL, timestamp TEXT ); -- Double-entry Ledger
entries CREATE TABLE ledger_entries ( id INTEGER PRIMARY KEY AUTOINCREMENT, trade_id TEXT,
account_code TEXT, debit REAL, credit REAL, description TEXT ); -- Daily P&L summary CREATE
TABLE daily_pnl ( day INTEGER, symbol TEXT, realized_pnl REAL, total_fees REAL, net_pnl
REAL, trade_count INTEGER );
```




Learning Resources



Official sqlite3 Module Docs

python.org — Full reference



SQLite Tutorial

Comprehensive SQLite tutorial site



W3Schools SQL

Interactive SQL reference



Real Python: sqlite3 Guide

Practical Python + SQLite



Common Mistake

⚠ Forgetting to commit: `conn.commit()` is required after INSERT/UPDATE/DELETE. Without it, your changes disappear when the connection closes. The evaluation engine will find zero trades in the ledger and fail you.

5 / 10

HTTP & JSON

The mock exchange is an HTTP REST API. You fetch data with GET, execute trades with POST, and parse everything as JSON.



REST API Endpoints

METHOD	ENDPOINT	PURPOSE
GET	/v1/ticker?symbol=BTC/USD	Latest price and 24h stats
GET	/v1/orderbook?symbol=ETH/USDT	5-level order book with bids/asks
POST	/v1/execute	Execute a trade (JSON body)
GET	/v1/account	Current account balances
GET	/v1/health	Server status



Making Requests

```
import urllib.request import json # GET request url = "http://localhost:8080/v1/ticker?symbol=BTC/USD" with urllib.request.urlopen(url, timeout=5) as resp: data = json.loads(resp.read().decode()) print(data) # {'symbol': 'BTC/USD', 'price': 67306.92, ...} # POST request (execute trade) body = json.dumps({ "symbol": "BTC/USD", "side": "buy", "quantity": 0.5, "type": "market" }).encode() req = urllib.request.Request("http://localhost:8080/v1/execute", data=body, headers={"Content-Type": "application/json"}) with urllib.request.urlopen(req, timeout=5) as resp: result = json.loads(resp.read().decode())
```



Learning Resources

**Official urllib.request Docs**

python.org — API reference

**Real Python: urllib Guide**

Urllib tutorial with examples

**MDN: HTTP Request Methods**

GET, POST, status codes explained

**Introducing JSON**

json.org — JSON specification

**Official json Module Docs**

python.org — json.loads/dumps

**HTTP Status Code Reference**

Every status code explained



Exchange Response Format

The mock exchange returns JSON. Here's what you'll parse:

```
// GET /v1/ticker?symbol=BTC/USD { "symbol": "BTC/USD", "price": 67306.92,
"change_24h_pct": -4.07, "volume_24h": 5739.1, "timestamp": "2026-06-18T22:53:26" } // GET
/v1/orderbook?symbol=ETH/USDT { "symbol": "ETH/USDT", "bids": [{"price": 3142.89,
"quantity": 40.45}, ...], "asks": [{"price": 3151.16, "quantity": 40.45}, ...], "spread":
8.27, "mid": 3146.82 }
```

6 / 10

Command Line Basics

You interact with the Principal Strategist through a terminal. You must be comfortable navigating directories and running Python scripts.

 Essential Commands

COMMAND	WINDOWS	MACOS & LINUX
List files	<code>dir</code>	<code>ls -la</code>
Change directory	<code>cd folder</code>	<code>cd folder</code>
Print working dir	<code>cd</code>	<code>pwd</code>
Create directory	<code>mkdir folder</code>	<code>mkdir -p folder</code>
Run Python script	<code>python script.py</code>	<code>python3 script.py</code>
View file contents	<code>type file.txt</code>	<code>cat file.txt</code>

 Learning Resources



PowerShell Documentation
(Windows)

→

microsoft.com — Official guide



Linux Command Line for Beginners

→

ubuntu.com — Free tutorial



Codecademy: Learn Command Line

→

Interactive course (free tier)

7 / 10

Standard Library Tour

The Principal Strategizer uses **zero external packages**. Every module it imports ships with Python. Here's what you need to know about each one.

Modules You Will Use in Your Solutions

MODULE	WHAT YOU DO WITH IT	DOCS LINK
<code>urllib.request</code>	Send HTTP GET and POST requests to the mock exchange	docs
<code>urllib.error</code>	Handle <code>HTTPError</code> , <code>URLError</code> , and socket timeouts	docs
<code>json</code>	Parse exchange responses with <code>json.loads()</code> and encode request bodies with <code>json.dumps()</code>	docs
<code>sqlite3</code>	Connect to the ledger database, execute SQL queries, commit transactions	docs
<code>math</code>	<code>sqrt()</code> for standard deviation, <code>log()</code> for returns, trigonometric functions if needed	docs
<code>statistics</code>	<code>mean()</code> , <code>stdev()</code> , <code>variance()</code> for risk metrics (Days 16+)	docs
<code>time</code>	<code>time.sleep()</code> between polling intervals and backoff delays	docs
<code>typing</code>	<code>List</code> , <code>Dict</code> , <code>Optional</code> , <code>Tuple</code> , <code>Union</code> for type annotations	docs

Modules Used By the Simulator (Understanding Only)

You don't need to master these, but understanding them helps:

MODULE	ROLE IN THE SIMULATOR
<code>cmd</code>	Powers the CLI <code>strategist></code> prompt and command parsing
<code>http.server</code>	Runs the mock exchange server on port 8080 and web dashboard on 8081
<code>socketserver</code>	Threaded mixin allowing multiple simultaneous connections

MODULE	ROLE IN THE SIMULATOR
threading	Runs both HTTP servers in background daemon threads
ast	Parses your code into an abstract syntax tree for the audit
subprocess	Runs your solution in an isolated child process during EOD
re	Regular expressions for text processing in the auditor
textwrap	Formatting CLI output with proper indentation
random	Generates realistic price movements in the mock exchange

Full Python Standard Library Reference



Python Standard Library

Complete index of every module

→



Python Module Index

A-Z of every built-in module

→

Financial Concepts

The curriculum assumes you understand (or can learn) these concepts as you go. Each is introduced in the daily briefings with full context.



Concepts by Phase

PHASE	CONCEPTS	DAY INTRODUCED
Phase 1	Simple Moving Average (SMA), Exponential Moving Average (EMA), Bollinger Bands, Relative Strength Index (RSI), Variance, Standard Deviation, Sharpe Ratio, Max Drawdown	Days 1–20
Phase 2	Cross-exchange spread, Triangular arbitrage cycles, Kelly Criterion, Position sizing, Slippage, Fill probability, Latency modeling	Days 31–55
Phase 3	Value at Risk (VaR), Conditional VaR (CVaR / Expected Shortfall), Stress testing, Scenario analysis, Financial instrument classification, Impairment testing	Days 61–75



Free Learning Resources (by Concept)

SMA / EMA

Investopedia: Moving Average →
Investopedia: EMA →

Bollinger Bands

Investopedia: Bollinger Bands →
Investopedia: RSI →

Risk Metrics

Investopedia: Sharpe Ratio →
Investopedia: Max Drawdown →

Arbitrage

Investopedia: Arbitrage →
Binance: Triangular Arbitrage →

Position Sizing

Investopedia: Kelly Criterion →
How to Apply Kelly Criterion →

VaR & CVaR

Investopedia: Value at Risk →
Investopedia: CVaR →



Key Formulas You'll Implement

```
# SMA - Simple Moving Average def sma(prices: List[float], window: int) ->
List[Optional[float]]: result: List[Optional[float]] = [] for i in range(len(prices)): if i
< window - 1: result.append(None) else: result.append(sum(prices[i-window+1:i+1]) / window)
return result # Sharpe Ratio (annualized) def sharpe_ratio(returns: List[float],
risk_free_rate: float = 0.0) -> float: excess_returns = [r - risk_free_rate for r in
returns] return (statistics.mean(excess_returns) / statistics.stdev(excess_returns)) *
math.sqrt(252) # Kelly Criterion (optimal bet size) def kelly_fraction(win_prob: float,
win_loss_ratio: float) -> float: return (win_prob * win_loss_ratio - (1 - win_prob)) /
win_loss_ratio
```


CMA, SOCPA & IFRS Standards

The simulator enforces three real-world professional accounting frameworks. You don't need to be a certified accountant, but you must understand what each standard requires of your code.

🏛️ CMA — Certified Management Accountant

The **Institute of Management Accountants (IMA)** CMA certification focuses on financial planning, analysis, control, and decision support. In the simulator, CMA standards apply to how you structure your data pipelines and financial calculations.

SECTION	RELEVANCE	SIMULATOR APPLICATION
Part 1, Section C	Financial Statement Analysis	Analyzing order book data, calculating indicators
Part 1, Section E	Investment Decisions	Position sizing, risk-reward evaluation
Part 2, Section A	Financial Statement Analysis	P&L reporting, ledger reconciliation
Part 2, Section D	Risk Management	VaR, CVaR, stress testing, exposure limits

- IMA CMA PROGRAM →
- INVESTOPEDIA: CMA OVERVIEW →

🏛️ SOCPA — Saudi Organization for Chartered and Professional Accountants

SOCPA sets accounting, auditing, and governance standards in Saudi Arabia. In the simulator, SOCPA standards apply to your audit trails, documentation requirements, and trade recording procedures.

STANDARD	RELEVANCE	SIMULATOR APPLICATION
Standard 4	Documentation & audit trails	Every trade must have complete provenance: timestamps, authorization, execution details
Standard 7	Financial instruments	Classification of crypto assets, measurement at fair value
Standard 12	Revenue recognition	Trade revenue recorded correctly, matched with fees

- SOCPA OFFICIAL SITE →
- SOCPA STANDARDS →

► **Practical Tip:** In the simulator, SOCPA compliance means **logging everything** — every trade, every calculation, every decision. If it's not in the ledger, it didn't happen.

 IFRS — International Financial Reporting Standards

IFRS provides global accounting standards. In the simulator, IFRS 9 (Financial Instruments) is the most relevant standard for how you classify and report trading assets.

STANDARD	RELEVANCE	SIMULATOR APPLICATION
IFRS 9	Financial Instruments	Classification (FVTPL, FVTOCI, amortized cost), impairment, hedge accounting
IFRS 13	Fair Value Measurement	Hierarchy levels (Level 1/2/3), disclosure requirements
IFRS 15	Revenue from Contracts	When and how to recognize trade revenue

- IFRS 9 FULL TEXT →
- IFRS 13 FULL TEXT →
- INVESTOPEDIA: IFRS OVERVIEW →

Environment Setup

Everything you need to install and configure before running the simulator. **Spoiler: it's almost nothing.**

⚙️ System Requirements

REQUIREMENT	MINIMUM	RECOMMENDED
Python version	3.8	3.11 – 3.13
RAM	128 MB	512 MB+
Disk space	50 MB	100 MB
OS	Any (Windows/macOS/Linux)	—
Network	localhost only (ports 8080 + 8081)	—

🔗 Verification Links



Download Python
python.org — Latest versions

→



Python on Windows Guide
Installation & PATH setup

→



Python Package Installation
pip tutorial (not needed, but good to know)

→

⚡ Quick Verification

Open a terminal and run these commands to confirm your environment is ready:

```
# Check Python version python --version # Expected: Python 3.8+ # Verify ports are free #  
(nothing should be listening on 8080 or 8081) # Test import of key modules python -c  
"import sqlite3, json, urllib, ast, http.server, cmd, threading; print('All modules OK')"
```

✓ FINAL CHECKLIST

Are You Ready?

Mark off each item before you type start for the first time.

- | | |
|--|--|
| ☐ I can write Python functions with type annotations | ☐ I understand try/except/else/finally for all network calls |
| ☐ I can use <code>urllib.request</code> for GET and POST requests | ☐ I can parse JSON with <code>json.loads()</code> and encode with <code>json.dumps()</code> |
| ☐ I can execute SQL queries via <code>sqlite3</code> (INSERT, SELECT, CREATE TABLE) | ☐ I know to call <code>conn.commit()</code> after writing to the database |
| ☐ I understand list slicing: <code>prices[-5:]</code> for sliding windows | ☐ I know what SMA, EMA, and Sharpe Ratio are conceptually |
| ☐ I have Python 3.8+ installed and working | ☐ Ports 8080 and 8081 are free on my machine |

✓ Checked all 10? You're ready to begin.

► Next step: `python arbitrage_academy.py`

Principal Strategist — Before You Begin Guide v1.0.0

[Training Program](#) · [Landing Page](#) · [Product Details](#) · [License](#) · [Dashboard](#)

All linked resources are free and publicly available. Python is a trademark of the Python Software Foundation.